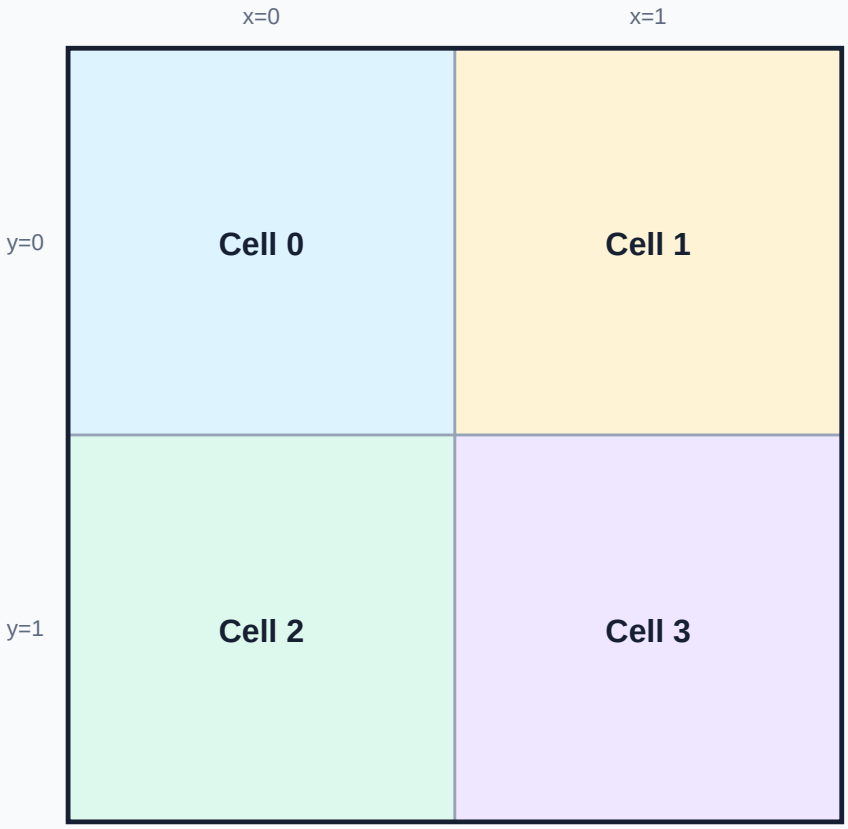
STEP 1

Begin with the familiar 2 x 2 uniform grid

At this point, a voxel and a cell are the same thing: each grid square owns its own primitive-reference list.



Ordinary grid lookup

```
index = x + y x width
```

For voxel (0, 1): index = 0 + 1 x 2 = 2, so the lookup returns Cell 2.

Cell 2 `min=(0,1), max=(1,2)`

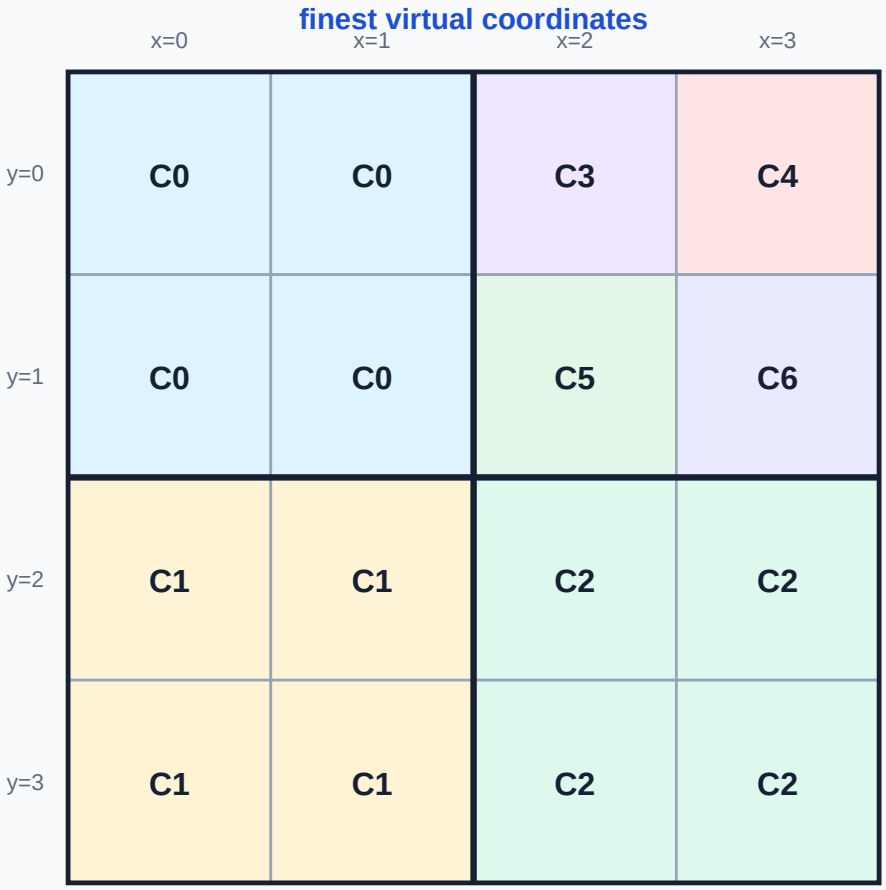
`references=[...]`

No hierarchy yet

STEP 2

Only the top-right region needs more detail

We split that one top-level square into four smaller cells. The other three regions stay large.



What actually changed?

Top-level grid	2 x 2
Finest coordinate lattice	4 x 4
Final compact cells	7

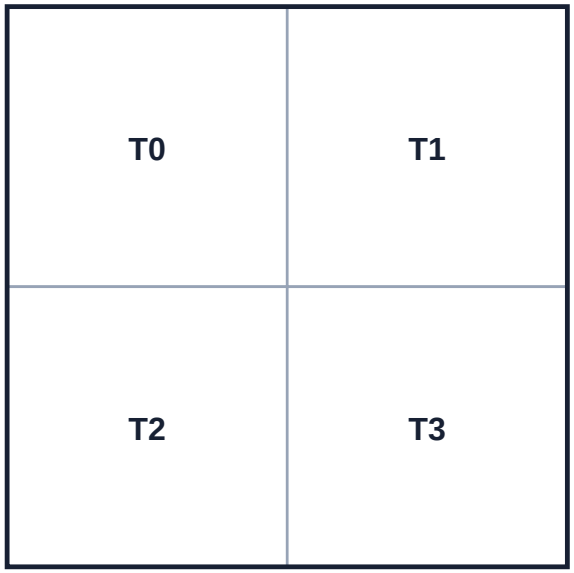
Build keeps coarse leaves first (C0-C2), then child leaves (C3-C6). The 4 x 4 lattice is only a shared ruler.

STEP 3

The whole scene gets a finer integer ruler

Subdivision introduces half-size boundaries. Hagrid scales the common coordinate system instead of storing fractional bounds.

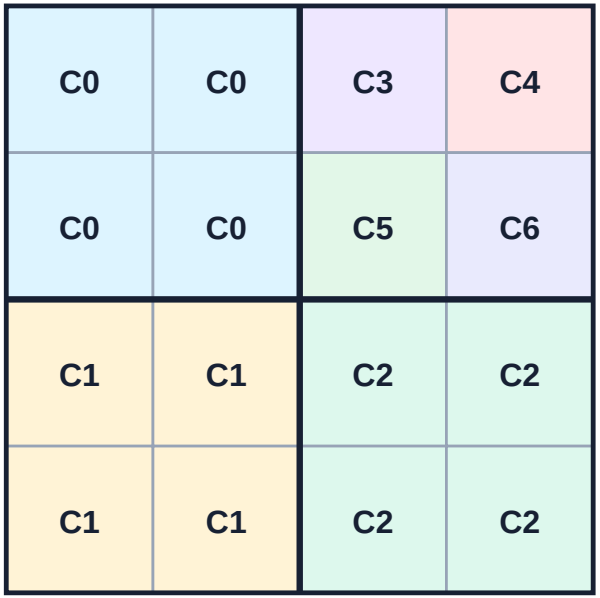
Before subdivision



temporary top cells T0-T3

scale x2

After subdivision + compaction



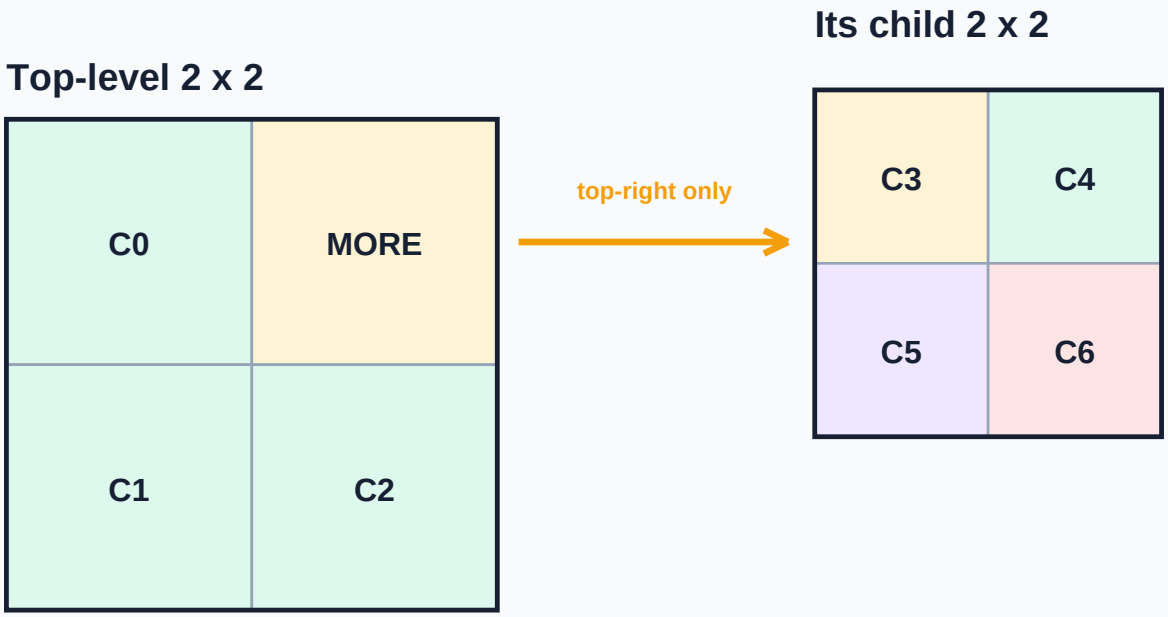
boundaries: 0, 1, 2, 3, 4

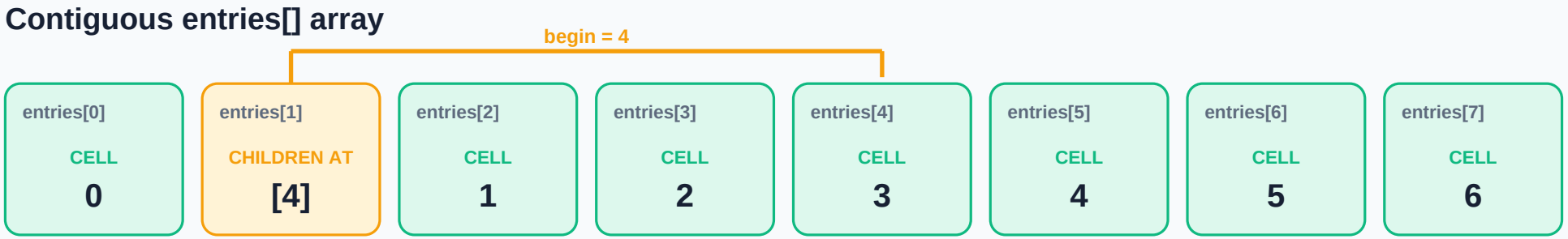
Large cells simply span several virtual voxels.

STEP 4

Store the nested grids in one flat entry array

Every entry says either STOP at a cell, or continue in a smaller grid whose entries begin elsewhere in the array.

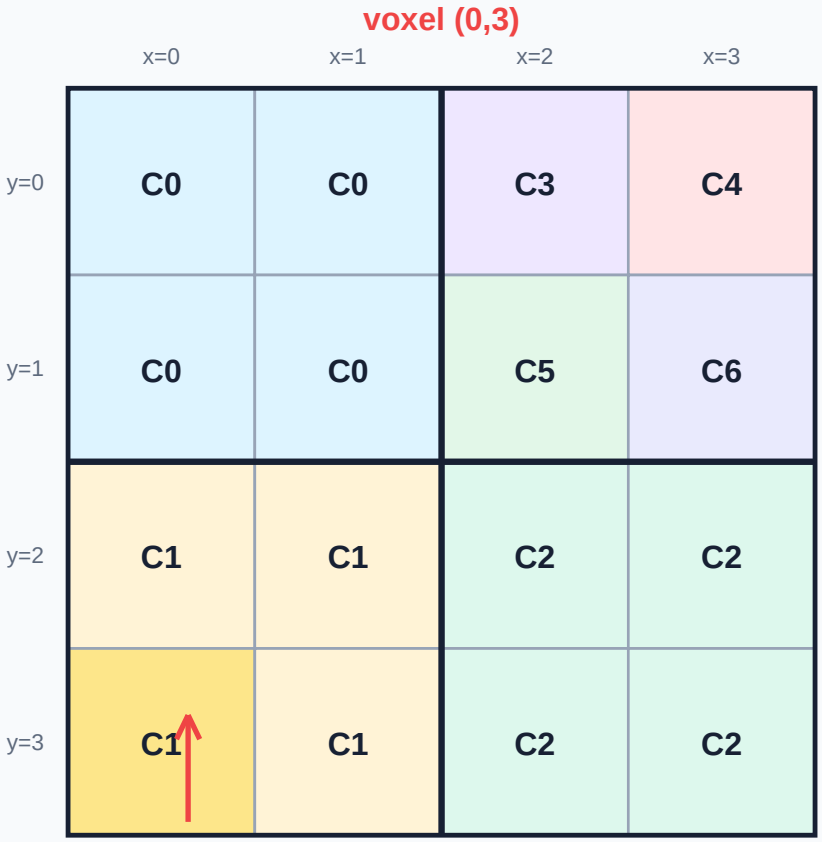




STEP 5

Trace lookup A: voxel (0, 3)

This voxel lies in a large, unsubdivided region, so the lookup ends after one array access.



1. Choose top-level square

```
topX = 0 / 2 = 0
topY = 3 / 2 = 1
```

`topIndex = 0 + 1 x 2 = 2`

2. Read entries[2]

entries[2]

CELL

1

STOP **Result: Cell 1**

STEP 6

Trace lookup B: voxel (3, 1)

This voxel lands in the refined top-right region, so one entry redirects the lookup to its four children.

A Top square: (3/2, 1/2) = (1,0)

entries[1]

CHILDREN AT

[4]

children start at 4

B Local square: (3%2, 1%2) = (1,1)

```
child offset = 1 + 1 x 2 = 3
```

child index = 4 + 3 = 7

entries[7]

CELL

6

STOP -> Cell 6

STEP 7

Refine C6 one more time

C6 is the bottom-right first-level region. In final compaction, six earlier leaves precede its four second-level leaves, so they receive C6-C9.

Before: 4 x 4 virtual lattice

C0	C0	C3	C4
C0	C0	C5	C6
C1	C1	C2	C2
C1	C1	C2	C2

refine C6

scale x2

After: 8 x 8 virtual lattice

C0	C0	C0	C0	C3	C3	C4	C4
C0	C0	C0	C0	C3	C3	C4	C4
C0	C0	C0	C0	C5	C5	C6	C7
C0	C0	C0	C0	C5	C5	C8	C9
C1	C1	C1	C1	C2	C2	C2	C2
C1	C1	C1	C1	C2	C2	C2	C2
C1	C1	C1	C1	C2	C2	C2	C2
C1	C1	C1	C1	C2	C2	C2	C2

Prefix-sum order: 6 earlier leaves, then the four new leaves C6-C9.

STEP 8

Read the complete 8 x 8 ownership map

The repeated labels show which fine virtual voxels belong to each actual cell. Thick lines mark the nested 2 x 2 subdivision boundaries.

	x=0	x=1	x=2	x=3	x=4	x=5	x=6	x=7
y=0	C0	C0	C0	C0	C3	C3	C4	C4
y=1	C0	C0	C0	C0	C3	C3	C4	C4
y=2	C0	C0	C0	C0	C5	C5	C6	C7
y=3	C0	C0	C0	C0	C5	C5	C8	C9
y=4	C1	C1	C1	C1	C2	C2	C2	C2
y=5	C1	C1	C1	C1	C2	C2	C2	C2
y=6	C1	C1	C1	C1	C2	C2	C2	C2
y=7	C1	C1	C1	C1	C2	C2	C2	C2

Cell bounds in virtual coordinates

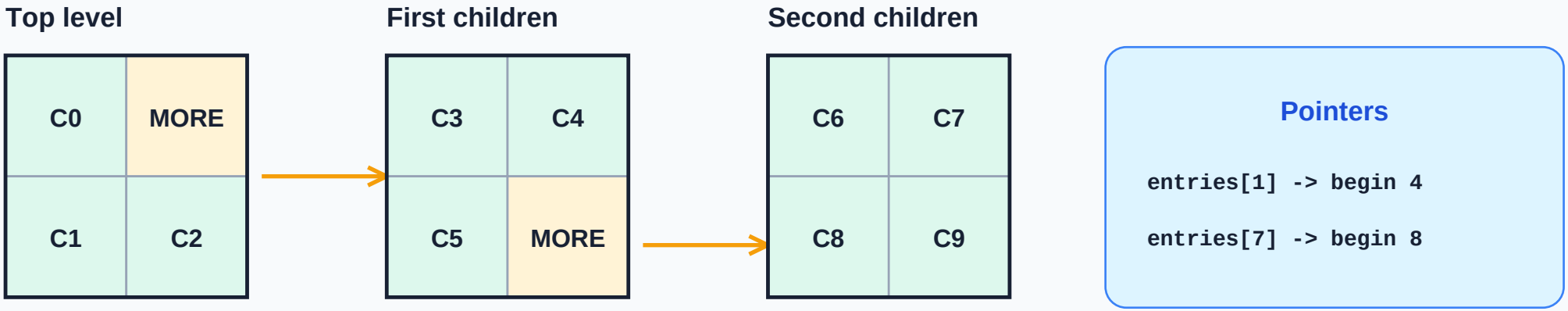
```
C0  min=(0,0)  max=(4,4)
C1  min=(0,4)  max=(4,8)
C2  min=(4,4)  max=(8,8)
C3  min=(4,0)  max=(6,2)
C4  min=(6,0)  max=(8,2)
C5  min=(4,2)  max=(6,4)
C6  min=(6,2)  max=(7,3)
C7  min=(7,2)  max=(8,3)
C8  min=(6,3)  max=(7,4)
C9  min=(7,3)  max=(8,4)
```

10 actual cells, not 64

STEP 9

Three nested 2 x 2 grids become one flat array

After compaction, every active cell has one consecutive ID from C0 through C9. The two CHILDREN entries are not cells.



entries[]

entries[0] CELL 0	entries[1] CHILDREN AT [4]	entries[2] CELL 1	entries[3] CELL 2	entries[4] CELL 3	entries[5] CELL 4
entries[6] CELL 5	entries[7] CHILDREN AT [8]	entries[8] CELL 6	entries[9] CELL 7	entries[10] CELL 8	entries[11] CELL 9

STEP 10

Trace voxel (7, 3) through both child levels

The top level uses division only. Each child level divides to its scale, then uses modulo 2 to select its local position.

	x=0	x=1	x=2	x=3	x=4	x=5	x=6	x=7
y=0	C0	C0	C0	C0	C3	C3	C4	C4
y=1	C0	C0	C0	C0	C3	C3	C4	C4
y=2	C0	C0	C0	C0	C5	C5	C6	C7
y=3	C0	C0	C0	C0	C5	C5	C8	C9
y=4	C1	C1	C1	C1	C2	C2	C2	C2
y=5	C1	C1	C1	C1	C2	C2	C2	C2
y=6	C1	C1	C1	C1	C2	C2	C2	C2
y=7	C1	C1	C1	C1	C2	C2	C2	C2

A Top level

`(7/4, 3/4) = (1,0)`

`entries[1] -> children begin 4`

B First child

`((7/2)%2, (3/2)%2) = (1,1)`

`offset 3 -> entries[7] -> begin 8`

C Second child

`(7%2, 3%2) = (1,1)`

`offset 3 -> entries[11] -> Cell 9`

voxel (7,3) -> Cell 9

STEP 11

The storage saving grows with depth

A dense lookup table reserves one ownership entry for every finest voxel. The hierarchy stores only top-level entries plus children for refined regions.

Dense ownership map

8 x 8

64 map entries

even where ownership repeats

Hierarchical ownership map

4	top entries
+ 4	first children
+ 4	second children

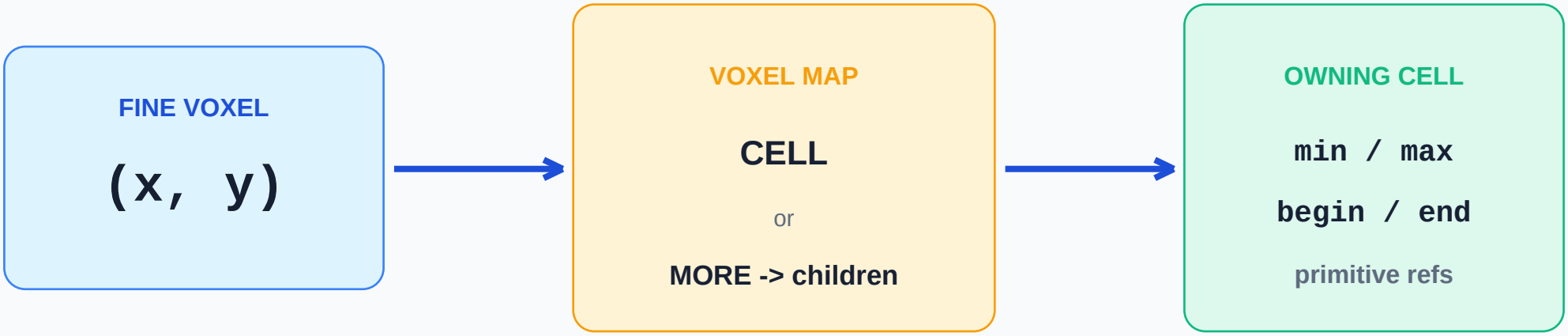
12 map entries

64 dense entries -> 12 hierarchical entries -> 10 actual cells

THE MENTAL MODEL

A fine address maps to an adaptive cell

The cells still have min/max bounds and primitive references. The voxel map is only the fast directory that finds the owning cell.



Subdivision increases global coordinate precision

but allocates new map entries only where finer detail is needed.